

Document number: P0958R0
Date: 2018-02-11
Project: Programming Language C++
Audience: SG1 - Concurrency and Parallelism
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to support proposed Executors TS

Introduction

The purpose of this paper is to illustrate the likely changes to the Networking TS to conform to the proposed Executors TS in P0443R5. At this time, this paper is intended only as an aid to discussion of P0443R5, and is not intended for review and incorporation into the Networking TS.

All changes are relative to N4711.

Summary of changes

This paper proposes the following changes to the Networking TS:

- [Add a reference to the executors proposal.](#)
- [Remove `executor\_work\_guard` and make `work\_guard`, as these have been superseded by the executors proposal's `execution::outstanding\_work\_property`.](#)
- [Remove the `Executor` type requirements and the `is\_executor` type trait.](#)
- [Specify the "Requirements on asynchronous operations" in terms of the new executors model.](#)
- [Update `system\_executor` and `system\_context` to conform to the new executors model.](#)
- [Remove the polymorphic wrapper `executor` and replace it with a type alias for the executors proposal's `execution::executor`.](#)
- [Specify the free functions `dispatch`, `post`, and `defer` in terms of the new executors model.](#)
- [Update the strand adapter to conform to the new executors model.](#)
- [Update the use `future\_completion` token to conform to the new executors model.](#)
- [Update `io\_context` to conform to the new executors model.](#)
- [Add `context\(\)` member functions to I/O objects as a convenience.](#)

Proposed wording

Add a reference to the executors proposal

Add a reference to the executors proposal in -5- Namespaces and headers [namespaces]:

-2- Unless otherwise specified, references to other entities described in this Technical Specification are assumed to be qualified with `std::experimental::net::v1::`, references to entities described in the C++ standard are assumed to be qualified with `std::`, **and** references to entities described in C++ Extensions for Library Fundamentals are assumed to be qualified with `std::experimental::fundamentals_v2::`, and references to entities described in P0443R5 A Unified Executors Proposal for C++ are assumed to be qualified with `std::experimental::executors_v1::`.

Remove superseded `executor_work_guard` and make `work_guard`

Remove `executor_work_guard` from -12.1- Header `<experimental/netfwd>` synopsis [fwd.decl.synop]:

```
template<class Executor>
class executor_work_guard;
```

Remove `executor_work_guard` and `make_work_guard` from **-13.1- Header <experimental/executor> synopsis [async.synop]**:

```
template<class Executor>
class executor_work_guard;

// 13.17, make_work_guard:

template<class Executor>
executor_work_guard<Executor>
make_work_guard(const Executor& ex);
template<class ExecutionContext>
executor_work_guard<typename ExecutionContext::executor_type>
make_work_guard(ExecutionContext& ctx);
template<class T>
executor_work_guard<associated_executor_t<T>>
make_work_guard(const T& t);
template<class T, class U>
auto make_work_guard(const T& t, U& u)
-> decltype(make_work_guard(get_associated_executor(t, forward<U>(u))));
```

Remove sections **-13.16- Class template `executor_work_guard` [async.exec.work.guard]** and **-13.17- Function `make_work_guard` [async.make.work.guard]** in their entirety.

Remove Executor requirements and `is_executor` type trait

Modify **Table 3 - Template parameters and type requirements [summary]** as follows:

Executor	executor(13.2.2) general requirements on executors (P0443R5).
----------	---

Remove `is_executor` and `is_executor_v` from **-13.1- Header <experimental/executor> synopsis [async.synop]**:

```
template<class T> struct is_executor;

template<class T>
constexpr bool is_executor_v = is_executor<T>::value;
```

Remove section **-13.2.2- Executor requirements [async.reqmts.executor]** in its entirety.

Modify **Table 5 - ExecutionContext requirements [async.reqmts.executioncontext]** as follows:

expression	return type	assertion/note pre/post-condition
<code>X::executor_type</code>	type meeting Executor(13.2.2) requirements A type satisfying the general requirements on executors (P0443R5).	

Modify section **-13.2.7.8- I/O executor [async.reqmts.async.io.exec]** as follows.

-1- An asynchronous operation has an associated executor satisfying the ~~Executor(13.2.2) requirements~~ general requirements on executors (P0443R5). If not otherwise specified by the asynchronous operation, this associated executor is an object of type `system_executor`.

[...]

-3- Let Executor1 be the type of the associated executor. Let ex1 be a value of type Executor1, representing the associated executor object obtained as described above. execution::can_query_v<Executor1, execution::context t> shall be true, and execution::query(ex1, execution::context t) shall yield a value of type execution_context& or of type E&, where E satisfies the ExecutionContext (13.2.3) requirements.

Modify section -13.2.7.9- **Completion handler executor** [async.reqmts.async.handler.exec] as follows.

-1- A completion handler object of type CompletionHandler has an associated executor satisfying the ~~Executor requirements (13.2.2)~~general requirements on executors (P0443R5). The type of this associated executor is associated_executor_t<CompletionHandler, Executor1>. Let Executor2 be the type associated_executor_t<CompletionHandler, Executor1>. Let ex2 be a value of type Executor2 obtained by performing get_associated_executor(completion_handler, ex1). execution::can_query_v<Executor2, execution::context t> shall be true, and execution::query(ex2, execution::context t) shall yield a value of type execution_context& or of type E&, where E satisfies the ExecutionContext (13.2.3) requirements.

Modify section -13.2.7.14- **Composed asynchronous operations** [async.reqmts.async.composed] as follows.

An intermediate operation's completion handler shall have an associated executor that is either:

- the type Executor2 and object ex2 obtained from the completion handler type CompletionHandler and object completion_handler; or
- an object of an unspecified type satisfying the ~~Executor requirements (13.2.2)~~general requirements on executors (P0443R5), that delegates executor operations to the type Executor2 and object ex2.

Remove section -13.9- **Class template is_executor** [async.is.exec] in its entirety.

Modify section -13.10- **Executor argument tag** [async.executor.arg] as follows.

The executor_arg_t struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, types may have constructors with executor_arg_t as the first argument, immediately followed by an argument of a type that satisfies the ~~Executor requirements (13.2.2)~~general requirements on executors (P0443R5).

Modify section -13.12- **Class template associated_executor** [async.assoc.exec] as follows.

-1- Class template associated_executor is an associator (13.2.6) for ~~the Executor (13.2.2) type requirements executors~~, with default candidate type system_executor and default candidate object system_executor().

Modify **Table 9 - associated_executor specialization requirements** as follows:

Expression	Return type	Note
typename X::type	A type meeting Executor requirements (13.2.2) <u>A type satisfying the general requirements on executors (P0443R5).</u>	

Modify section -13.13- **Function get_associated_executor** [async.assoc.exec.get] as follows.

-3- **Remarks:** This function shall not participate in overload resolution unless is_executor_v<Executor> is trueis_convertible<Executor&, execution_context&>::value is false.

Modify section -13.14- **Class template executor_binder** [async.exec.binder] as follows.

-1- The class template executor_binder binds executors to objects. A specialization executor_binder<T, Executor> binds an executor of type Executor satisfying the ~~Executor requirements (13.2.2)~~general requirements on executors (P0443R5) to an object or function of type T.

Modify section -13.15- **Function bind_executor** [*async.bind.executor*] as follows.

-2- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~is_convertible<Executor&, execution_context&>::value is false.

Modify section -13.22- **Function dispatch** [*async.dispatch*] as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~is_convertible<Executor&, execution_context&>::value is false.

Modify section -13.23- **Function post** [*async.post*] as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~is_convertible<Executor&, execution_context&>::value is false.

Modify section -13.24- **Function defer** [*async.defer*] as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~is_convertible<Executor&, execution_context&>::value is false.

Modify section -13.25- **Class template strand** [*async.strand*] as follows.

-1- The class template strand is a wrapper around an object of type Executor satisfying the OneWayExecutor requirements (~~13.2.2~~P0443R5).

[...]

-2- strand<Executor> satisfies the ~~Executor-(13.2.2) requirements~~OneWayExecutor requirements (P0443R5).

Modify **Table 17 - AsyncReadStream requirements** [*buffer.stream.reqmts.asyncreadstream*] as follows:

operation	type	semantics, pre/post-conditions
a.get_executor()	A type satisfying the Executor requirements (13.2.2) <u>general requirements on executors (P0443R5).</u>	Returns the associated I/O executor.

Modify **Table 19 - AsyncWriteStream requirements** [*buffer.stream.reqmts.asyncwritestream*] as follows:

operation	type	semantics, pre/post-conditions
a.get_executor()	A type satisfying the Executor requirements (13.2.2) <u>general requirements on executors (P0443R5).</u>	Returns the associated I/O executor.

Specify the "Requirements on asynchronous operations" in terms of the new executors model

Modify section -13.2.7.10- **Outstanding work** [*async.reqmts.async.work*] as follows.

-1- Until the asynchronous operation has completed, the asynchronous operation shall maintain:

- ~~an object work1 of type executor_work_guard<Executor1>, initialized as work1(ex1), and where work1.owns_work() == true; and~~
- ~~an object work2 of type executor_work_guard<Executor2>, initialized as work2(ex2), and where work2.owns_work() == true;~~

- an executor object work1, initialized as `execution::prefer(ex1, execution::outstanding_work)`; and
- an executor object work2, initialized as `execution::prefer(ex2, execution::outstanding_work)`.

Modify section -13.2.7.12- *Execution of completion handler on completion of asynchronous operation* [*async.reqmts.async.completion*] as follows:

-3- If an asynchronous operation completes immediately (that is, within the thread of execution calling the initiating function, and before the initiating function returns), the completion handler shall be submitted for execution as if by performing ~~`ex2.post(std::move(f), alloc2)`~~. ~~Otherwise, the completion handler shall be submitted for execution as if by performing `ex2.dispatch(std::move(f), alloc2)`;~~

`execution::prefer(`
`execution::require(ex2, execution::oneway, execution::single, execution::never_blocking),`
`execution::allocator(alloc2)).execute(std::move(f));`

Otherwise, the completion handler shall be submitted for execution as if by performing:

`execution::prefer(`
`execution::require(work2, execution::oneway, execution::single),`
`execution::possibly_blocking, execution::allocator(alloc2)).execute(std::move(f));`

Update `system_executor` and `system_context` to conform to the new executors model

Remove `system_executor` from, and add `system_context` to, -12.1- Header `<experimental/netfwd>` synopsis [*fwd.decl.synop*]:

~~`class system_executor;`~~
`class system_context;`

Update `system_executor` to be a type alias in -13.1- Header `<experimental/executor>` synopsis [*async.synop*]:

~~`class system_executor;`~~
`class system_context;`
`using system_executor = system_context::executor_type;`

~~`bool operator==(const system_executor&, const system_executor&);`~~
~~`bool operator!=(const system_executor&, const system_executor&);`~~

Remove section -13.18- *Class `system_executor`* [*async.system.exec*] in its entirety.

Modify section -13.19- *Class `system_context`* [*async.system.context*] as follows:

-1- Class `system_context` implements ~~the execution context associated with `system_executor` objects~~an execution context that represents the ability to run a submitted function object on any thread.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class system_context : public execution_context
    {
    public:
        // types:

        using executor_type = system_executorsee below;

```

[...]

-2- The class `system_context` satisfies the `ExecutionContext` (13.2.3) type requirements.

-?- executor_type is an executor type conforming to the specification for system_context executor types described below. Executor objects of type executor_type have the following properties established:

- execution::oneway
- execution::single
- execution::possibly_blocking
- execution::not_continuation
- execution::thread_execution_mapping
- execution::allocator(std::allocator<void>(.)).

[...]

```
executor_type get_executor() noexcept;
```

-5- Returns: system_executor()executor_type().

After section -13.19- Class system_context [async.system.context] insert a new section as follows:

-13.- system_context executor types

-1- All executor types accessible through system_context::executor_type(), system_context::get_executor(), and subsequent calls to the member function require, conform to the following specification.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class C
    {
    public:
        // construct / copy / destroy:

        C() {}

        // executor operations:

        see below require(execution::never_blocking t) const;
        see below require(execution::possibly_blocking t) const;
        see below require(execution::continuation t) const;
        see below require(execution::not_continuation t) const;
        see below require(execution::allocator t<void>) const;
        template<class ProtoAllocator>
        see below require(const execution::allocator t<ProtoAllocator>& a) const;

        static constexpr bool query(execution::thread_execution_mapping t) noexcept;
        static system_context& query(execution::context t) noexcept;
        see below query(execution::allocator t<void>) const noexcept;
        template<class ProtoAllocator>
        see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

        template<class Function>
        void execute(Function&& f) const;
    };

    bool operator==(const C& a, const C& b) noexcept;
    bool operator!=(const C& a, const C& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std
```

-2- C is a type satisfying the OneWayExecutor requirements (P0443R5).

-13.?.1- system context executor operations

see below require(execution::never_blocking t) const;
see below require(execution::possibly_blocking t) const;
see below require(execution::continuation t) const;
see below require(execution::not_continuation t) const;

-1- **Returns:** A system context executor object of an unspecified type conforming to these specifications, with the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of *this.

see below require(execution::allocator t<void>) const;

-2- **Returns:** require(execution::allocator(std::allocator<void>(.)).).

template<class ProtoAllocator>
see below require(const execution::allocator t<ProtoAllocator>& a) const;

-3- **Returns:** A system context executor object of an unspecified type conforming to these specifications, with the execution::allocator t<ProtoAllocator> property established such that allocation and deallocation associated with function submission will be performed using a copy of a.value(.). All other properties of the returned executor object are identical to those of *this.

static constexpr bool query(execution::thread_execution_mapping t) noexcept;

-4- **Returns:** true.

static system context& query(execution::context t) const;

-5- **Returns:** A reference to the system context object.

see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

-6- **Returns:** The allocator object associated with the executor, with type and value as previously established by the execution::allocator t<ProtoAllocator> property.

template<class Function>
void execute(Function&& f) const

-7- **Effects:** Submits the function f for execution according to the OneWayExecutor requirements and the properties established for *this. If f exits via an exception, calls std::terminate().

-13.?.2- system context executor comparisons

bool operator==(const C& a, const C& b) noexcept;

-1- **Returns:** true.

bool operator!=(const C& a, const C& b) noexcept;

-2- **Returns:** false.

Remove the polymorphic wrapper executor and replace it with a type alias

Remove `executor` from **-12.1- Header** `<experimental/netfwd>` synopsis `[fwd.decl.synop]`:

```
class executor;
```

Remove classes `bad_executor` and `executor`, and add a new type alias `executor`, in **-13.1- Header** `<experimental/executor>` synopsis `[async.synop]`:

```
class bad_executor;  
  
class executor;  
  
bool operator==(const executor& a, const executor& b) noexcept;  
bool operator==(const executor& e, nullptr_t) noexcept;  
bool operator==(nullptr_t, const executor& e) noexcept;  
bool operator!=(const executor& a, const executor& b) noexcept;  
bool operator!=(const executor& e, nullptr_t) noexcept;  
bool operator!=(nullptr_t, const executor& e) noexcept;  
using executor = execution::executor<  
    execution::oneway_t,  
    execution::single_t,  
    execution::context_t,  
    execution::possibly_blocking_t,  
    execution::never_blocking_t,  
    execution::prefer_only<execution::outstanding_work>,  
    execution::prefer_only<execution::not_outstanding_work>,  
    execution::prefer_only<execution::continuation>,  
    execution::prefer_only<execution::not_continuation>>;  
  
[...]  
  
} // inline namespace v1  
} // namespace net  
} // namespace experimental  
  
template<class Allocator>  
struct uses_allocator<experimental::net::v1::executor, Allocator>  
    : true_type {};  
  
} // namespace std
```

Remove sections **-13.20- Class `bad_executor`** `[async.bad.exec]` and **-13.21- Class `executor`** `[async.executor]` in their entirety.

Specify the free functions `dispatch`, `post`, and `defer` in terms of the new executors model

Modify section **-13.22- Function `dispatch`** `[async.dispatch]` as follows:

-1- [Note: The function `dispatch` satisfies the requirements for an asynchronous operation (13.2.7), except for the requirement that the operation uses `post`the `execution::never_blocking_property` if it completes immediately. -- end note]

[...]

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.dispatch(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~
- Performs:


```
    execution::prefer(
        execution::require(ex, execution::oneway, execution::single),
        execution::possibly_blocking, execution::allocator(alloc)).execute(
            std::move(completion.completion_handler));
```

where ex is the result of get_associated_executor(completion.completion_handler), and alloc is the result of get_associated_allocator(completion.completion_handler).

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `execution::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor work guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- `w.reset()`;

```
    execution::prefer(
        execution::require(w, execution::oneway, execution::single),
        execution::possibly_blocking, execution::allocator(alloc)).execute(std::move(h));
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`;~~
- Performs:

```
    execution::prefer(
        execution::require(ex, execution::oneway, execution::single),
        execution::possibly_blocking, execution::allocator(alloc)).execute(std::move(f));
```

Modify section -13.23- **Function post [async.post]** as follows:

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.post(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~
- Performs:

```
    execution::prefer(
        execution::require(ex, execution::oneway, execution::single, execution::never_blocking),
        execution::allocator(alloc)).execute(std::move(completion.completion_handler));
```

where ex is the result of get_associated_executor(completion.completion_handler), and alloc is the result of get_associated_allocator(completion.completion_handler).

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `execution::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

`execution::prefer(`
`execution::require(w, execution::oneway, execution::single),`
`execution::possibly_blocking, execution::allocator(alloc)).execute(std::move(h));`

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.~~
- Performs:

`execution::prefer(`
`execution::require(ex, execution::oneway, execution::single, execution::never_blocking),`
`execution::allocator(alloc)).execute(std::move(f));`

Modify section -13.24- **Function `defer` [`async.defer`]** as follows:

-1- [Note: The function `defer` satisfies the requirements for an asynchronous operation (13.2.7), except for the requirement that the operation uses ~~`post-if it completes immediately`~~. --end note]

[...]

-3- **Effects:**

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.defer(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~
- Performs:

`execution::prefer(`
`execution::require(ex, execution::oneway, execution::single, execution::never_blocking),`
`execution::continuation, execution::allocator(alloc)).execute(`
`std::move(completion.completion_handler));`

where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

[...]

-6- **Effects:**

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `execution::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,

- ~~an `executor_work_guard` object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
execution::prefer(
  execution::require(w, execution::oneway, execution::single),
  execution::possibly_blocking, execution::allocator(alloc)).execute(std::move(h)).;
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion_completion_handler)` prior to the construction of `f`.~~
- Performs:

```
  execution::prefer(
    execution::require(ex, execution::oneway, execution::single, execution::never_blocking),
    execution::continuation, execution::allocator(alloc)).execute(std::move(f)).;
```

Update the strand adapter to conform to the new executors model

Modify section -13.25- Class template `strand` [`async.strand`] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Executor>
    class strand
    {
    public:
```

[...]

```
    execution_context& context() const noexcept;
```

```
    void on_work_started() const noexcept;
```

```
    void on_work_finished() const noexcept;
```

```
    template<class Func, class ProtoAllocator>
    void dispatch(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Func, class ProtoAllocator>
    void post(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Func, class ProtoAllocator>
    void defer(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Property>
    see below require(const Property& p) const;
```

```
    template<class Property>
    see below query(const Property& p) const;
```

```
    template<class Function>
    void execute(Function&& f) const;
```

[...]

-3- A strand provides guarantees of ordering and non-concurrency. Given:

- strand objects `s1` and `s2` such that `s1 == s2`

- a function object `f1` ~~added~~submitted to the strand `s1` ~~using post or defer, or using dispatch~~when the execution::never_blocking_property is established in s1, or when `s1.running_in_this_thread()` is `false`
- a function object `f2` ~~added~~submitted to the strand `s2` ~~using post or defer, or using dispatch~~when the execution::never_blocking_property is established in s2, or when `s2.running_in_this_thread()` is `false`

Modify section -13.25.4- **strand operations** [*async.strand.ops*] as follows:

```
bool running_in_this_thread() const noexcept;
```

-2- Returns: `true` if the current thread of execution is running a function that was submitted to the strand, or to any other strand object ~~s such that s == *this, using dispatch, post or defer~~that shares the same ordered, non-concurrent state, using execute; otherwise `false`. [Note: That is, the current thread of execution's call chain includes a function that was submitted to the strand. --end note]

```
void on_work_started() const noexcept;
```

~~**-4- Effects:** Calls `inner_ex.on_work_started()`.~~

```
void on_work_finished() const noexcept;
```

~~**-5- Effects:** Calls `inner_ex.on_work_finished()`.~~

```
template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a) const;
```

~~**-6- Effects:** If `running_in_this_thread()` is `true`, calls `DECAY_COPY(forward<Func>(f))()` (C++ 2014 [thread.decaycopy]). [Note: If `f` exits via an exception, the exception propagates to the caller of `dispatch()`. --end note] Otherwise, requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex`, such that the guarantees of ordering and non-concurrency are met.~~

```
template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a) const;
```

~~**-7- Effects:** Requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex`, such that the guarantees of ordering and non-concurrency are met.~~

```
template<class Func, class ProtoAllocator> void defer(Func&& f, const ProtoAllocator& a) const;
```

~~**-8- Effects:** Requests invocation of `f`, as if by forwarding the function object `f` and allocator `a` to the executor `inner_ex`, such that the guarantees of ordering and non-concurrency are met.~~

```
template<class Property>
see below require(const Property& p) const;
```

-?- Returns: A strand `s` of type `strand<decay_t<decltype(inner_ex.require(p))>>`, where `s.inner_ex` is initialized with `inner_ex.require(p)`, and sharing the same ordered, non-concurrent state as `*this`.

-?- Remarks: Shall not participate in overload resolution unless `inner_ex.require(p)` is well-formed.

```
template<class Property>
see below query(const Property& p) const;
```

-?- Returns: `inner_ex.query(p)`.

-?- Remarks: Shall not participate in overload resolution unless `inner_ex.query(p)` is well-formed.

```
template<class Function>
void execute(Function&& f) const;
```

-?- **Effects:** Submits `f` to the executor `inner_ex`, such that the guarantees of ordering and non-concurrency are met.

Update the `use_future` completion token to conform to the new executors model

Modify section -13.26.2- `use_future_t` members [`async.use.future.members`] as follows:

-8- For any executor type `E`, the associated object for the associator `associated_executor<H, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown is caught by a function object and stored in the associated shared state.

Modify section -13.26.3- *Partial class template specialization `async_result` for `use_future_t` [`async.use.future.result`]* as follows:

-3- The implementation specializes `associated_executor` for `F`. For function objects executed using the associated executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown is caught by the executor and stored in the associated shared state.

-4- For any executor type `E`, the associated object for the associator `associated_executor<F, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown by a function object is caught by the executor and stored in the associated shared state.

Update `io_context` to conform to the new executors model

Modify section -14.2- *Class `io_context` [`io_context.io_context`]* as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class io_context : public execution_context
    {
    public:
        // types:

        class executor_type;
        using executor_type = see below;
    };
}
```

[...]

-1- The class `io_context` satisfies the `ExecutionContext` type requirements (13.2.3).

-?- `executor_type` is an executor type conforming to the specification for `io_context` executor types described below.

[...]

-4- For an object of type `io_context`, *outstanding work* is defined as the sum of:

- ~~the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any executor of the `io_context`;~~
- the number of existing executor objects associated with the `io_context` for which the `execution::outstanding_work` property is established;
- the number of function objects that have been added to the `io_context` via any executor of the `io_context`, but not yet executed; and
- the number of function objects that are currently being executed by the `io_context`.

[...]

```
executor_type get_executor() noexcept;
```

-3- **Returns:** An executor that may be used for submitting function objects to the `io_context`. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::single`
- `execution::possibly_blocking`
- `execution::not_continuation`
- `execution::not_outstanding_work`
- `execution::thread_execution_mapping`
- `execution::allocator(std::allocator<void>())`

[...]

-13- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

[...]

-19- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

[...]

-25- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

Remove section -14.3- Class `io_context::executor_type` [`io_context.exec`] in its entirety.

After section -14.2- Class `io_context` [`io_context.io_context`] insert a new section as follows:

-14.- io context executor types

All executor types accessible through `io_context::get_executor()`, and subsequent calls to the member function `require`, conform to the following specification.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class C
    {
    public:
        // construct / copy / destroy:

        C(const C& other) noexcept;
        C(C&& other) noexcept;

        C& operator=(const C& other) noexcept;
        C& operator=(C&& other) noexcept;

        // executor operations:

        see below require(execution::never_blocking t) const;
        see below require(execution::possibly_blocking t) const;
        see below require(execution::continuation t) const;
        see below require(execution::not_continuation t) const;
        see below require(execution::outstanding_work t) const;
        see below require(execution::not_outstanding_work t) const;
```

```

see below require(execution::allocator t<void>) const;
template<class ProtoAllocator>
see below require(const execution::allocator t<ProtoAllocator>& a) const;

static constexpr bool query(execution::thread execution_mapping t) noexcept;
io context& query(execution::context t) const noexcept;
see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

bool running_in_this_thread() const noexcept;

template<class Function>
void execute(Function&& f) const;
};

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-1- C is a type satisfying the OneWayExecutor requirements (P0443R5). Objects of type C are associated with an io context, and function objects submitted using the execute member function will be executed by the io context from within a run function.

-14.?.1- io context executor constructors

C(const C& other) noexcept;

-1- Postconditions: *this == other.

C(C&& other) noexcept;

-2- Postconditions: *this is equal to the prior value of other.

-14.?.2- io context executor assignment

C& operator=(const C& other) noexcept;

-1- Postconditions: *this == other.

-2- Returns: *this.

C& operator=(C&& other) noexcept;

-3- Postconditions: *this is equal to the prior value of other.

-4- Returns: *this.

-14.?.3- io context executor operations

see below require(execution::never_blocking t) const;
see below require(execution::possibly_blocking t) const;
see below require(execution::continuation t) const;
see below require(execution::not_continuation t) const;
see below require(execution::outstanding_work t) const;
see below require(execution::not_outstanding_work t) const;

-1- **Returns:** An executor object of an unspecified type conforming to these specifications, associated with the same io context as *this, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of *this.

see below `require(execution::allocator t<void>) const;`

-2- **Returns:** `require(execution::allocator(std::allocator<void>(.))).`

template<class ProtoAllocator>

see below `require(const execution::allocator t<ProtoAllocator>& a) const;`

-3- **Returns:** An executor object of an unspecified type conforming to these specifications, associated with the same io context as *this, with the `execution::allocator t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.value()`. All other properties of the returned executor object are identical to those of *this.

static constexpr bool query(execution::thread execution mapping t) noexcept;

-4- **Returns:** true.

io_context& query(execution::context t) const noexcept;

-5- **Returns:** A reference to the associated io context object.

see below `query(execution::allocator t<void>) const noexcept;`

template<class ProtoAllocator>

see below `query(const execution::allocator t<ProtoAllocator>&) const noexcept;`

-6- **Returns:** The allocator object associated with the executor, with type and value as previously established by the `execution::allocator t<ProtoAllocator>` property.

bool running_in_this_thread() const noexcept;

-7- **Returns:** true if the current thread of execution is calling a run function of the associated io context object. [Note: That is, the current thread of execution's call chain includes a run function. --end note]

template<class Function>

void execute(Function&& f) const

-8- **Effects:** Submits the function `f` for execution on the io context according to the `OneWayExecutor` requirements and the properties established for *this. If `f` exits via an exception, the exception does not propagate to the caller of `execute()`, but is instead subsequently propagated to a caller of a run function for the io context object.

-14.?4- io context executor comparisons

bool operator==(const C& a, const C& b) noexcept;

-1- **Returns:** `addressof(a.query(execution::context t)) == addressof(b.query(execution::context t)).`

bool operator!=(const C& a, const C& b) noexcept;

-2- **Returns:** `!(a == b).`

Add context() member functions to I/O objects as a convenience

Modify section -15.4- Class template `basic_waitable_timer` [timer.waitable] as follows:

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Clock, class WaitTraits = wait_traits<Clock>>
    class basic_waitable_timer
    {
    public:

```

[...]

```

// 15.4.4, basic_waitable_timer operations:

```

```

    io_context& context() noexcept;

```

```

    executor_type get_executor() noexcept;

```

[...]

```

basic_waitable_timer(io_context& ctx, const time_point& t);

```

-2- Postconditions:

- addressof(context()) == addressof(ctx).

[...]

```

basic_waitable_timer(io_context& ctx, const duration& d);

```

-3- **Effects:** Sets the expiry time as if by calling `expires_after(d)`.

-4- **Postconditions:** ~~`get_executor() == ctx.get_executor();`~~

- addressof(context()) == addressof(ctx).
- get_executor() == ctx.get_executor().

[...]

```

basic_waitable_timer(basic_waitable_timer&& rhs);

```

-5- **Effects:** Move constructs an object of class `basic_waitable_timer<Clock, WaitTraits>` that refers to the state originally represented by `rhs`.

-6- Postconditions:

- addressof(context()) == addressof(rhs.context()).

[...]

```

basic_waitable_timer& operator=(basic_waitable_timer&& rhs);

```

-1- **Effects:** Cancels any outstanding asynchronous operations associated with `*this` as if by calling `cancel()`, then moves into `*this` the state originally represented by `rhs`.

-2- Postconditions:

- addressof(context()) == addressof(rhs.context()).

[...]

-15.4.4- basic_waitable_timer operations [timer.waitable.ops]

io_context& context(.) noexcept;

-?- Returns: The associated execution context.

Modify section -18.6- Class template basic_socket [socket.basic] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Protocol>
    class basic_socket
    {
    public:
```

[...]

```
// 18.6.4, basic_socket operations:
```

io_context& context(.) noexcept;

```
    executor_type get_executor() noexcept;
```

[...]

```
explicit basic_socket(io_context& ctx);
```

-1- Postconditions:

- addressof(context(.)) == addressof(ctx).

[...]

```
basic_socket(io_context& ctx, const protocol_type& protocol);
```

-2- **Effects:** Opens this socket as if by calling open(protocol).

-3- Postconditions:

- addressof(context(.)) == addressof(ctx).

[...]

```
basic_socket(io_context& ctx, const endpoint_type& endpoint);
```

-4- **Effects:** Opens and binds this socket as if by calling:

```
open(endpoint.protocol());
bind(endpoint);
```

-5- Postconditions:

- addressof(context(.)) == addressof(ctx).

[...]

```
basic_socket(io_context& ctx, const protocol_type& protocol,
             const native_handle_type& native_socket);
```

-6- **Requires:** `native_socket` is a native handle to an open socket.

-7- **Effects:** Assigns the existing native socket into this socket as if by calling `assign(protocol, native_socket)`.

-8- **Postconditions:**

- `addressof(context()) == addressof(ctx).`

[...]

```
basic_socket(basic_socket&& rhs);
```

-9- **Effects:** Move constructs an object of class `basic_socket<Protocol>` that refers to the state originally represented by `rhs`.

-10- **Postconditions:**

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
template<class OtherProtocol>
    basic_socket(basic_socket<OtherProtocol>&& rhs);
```

-11- **Requires:** `OtherProtocol` is implicitly convertible to `Protocol`.

-12- **Effects:** Move constructs an object of class `basic_socket<Protocol>` that refers to the state originally represented by `rhs`.

-13- **Postconditions:**

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
basic_socket& operator=(basic_socket&& rhs);
```

-1- **Effects:** If `is_open()` is true, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by `rhs`.

-2- **Postconditions:**

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
template<class OtherProtocol>
    basic_socket& operator=(basic_socket<OtherProtocol>&& rhs);
```

-4- **Requires:** `OtherProtocol` is implicitly convertible to `Protocol`.

-5- **Effects:** If `is_open()` is true, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by `rhs`.

-6- Postconditions:

- [addressof\(context\(\)\) == addressof\(rhs.context\(\)\).](#)

[...]

-18.6.4- basic_socket operations [socket.basic.ops]

[io_context& context\(\) noexcept;](#)

[-?- Returns: The associated execution context.](#)

Modify section -18.9- Class template basic_socket_acceptor [socket.acceptor] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Protocol>
    class basic_socket_acceptor
    {
    public:
```

[...]

```
        // 18.9.4, basic_socket_acceptor operations:
```

```
        io\_context& context\(\) noexcept;
```

```
        executor_type get_executor() noexcept;
```

[...]

```
explicit basic_socket_acceptor(io_context& ctx);
```

-1- Postconditions:

- [addressof\(context\(\)\) == addressof\(ctx\).](#)

[...]

```
basic_socket_acceptor(io_context& ctx, const protocol_type& protocol);
```

-2- Effects: Opens this acceptor as if by calling open(protocol).

-3- Postconditions:

- [addressof\(context\(\)\) == addressof\(ctx\).](#)

[...]

```
basic_socket_acceptor(io_context& ctx, const endpoint_type& endpoint,
                     bool reuse_addr = true);
```

-4- Effects: Opens and binds this acceptor as if by calling:

```
open(endpoint.protocol());
if (reuse_addr)
    set_option(reuse_address(true));
bind(endpoint);
```

-5- Postconditions:

- `addressof(context()) == addressof(ctx).`

[...]

```
basic_socket_acceptor(io_context& ctx, const protocol_type& protocol,  
                      const native_handle_type& native_acceptor);
```

-6- Requires: `native_acceptor` is a native handle to an open acceptor.

-7- Effects: Assigns the existing native acceptor into this acceptor as if by calling `assign(protocol, native_acceptor)`.

-8- Postconditions:

- `addressof(context()) == addressof(ctx).`

[...]

```
basic_socket_acceptor(basic_socket_acceptor&& rhs);
```

-9- Effects: Move constructs an object of class `basic_socket_acceptor<Protocol>` that refers to the state originally represented by `rhs`.

-10- Postconditions:

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
template<class OtherProtocol>  
basic_socket_acceptor(basic_socket_acceptor<OtherProtocol>&& rhs);
```

-11- Requires: `OtherProtocol` is implicitly convertible to `Protocol`.

-12- Effects: Move constructs an object of class `basic_socket_acceptor<Protocol>` that refers to the state originally represented by `rhs`.

-13- Postconditions:

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
basic_socket_acceptor& operator=(basic_socket_acceptor&& rhs);
```

-1- Effects: If `is_open()` is true, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX `close(native_handle())`. Then moves into `*this` the state originally represented by `rhs`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true.

-2- Postconditions:

- `addressof(context()) == addressof(rhs.context()).`

[...]

```
template<class OtherProtocol>  
basic_socket_acceptor& operator=(basic_socket_acceptor<OtherProtocol>&& rhs);
```

-4- **Requires:** OtherProtocol is implicitly convertible to Protocol.

-5- **Effects:** If is_open() is true, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX close(native_handle()). Then moves into *this the state originally represented by rhs. Completion handlers for canceled operations are passed an error code ec such that ec == errc::operation_canceled yields true.

-6- **Postconditions:**

- addressof(context()) == addressof(rhs.context()).

[...]

-18.9.4- basic_socket_acceptor operations [socket.acceptor.ops]

io_context& context() noexcept;

-?- Returns: The associated execution context.

Modify section -21.17- Class template ip::basic_resolver [internet.resolver] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

    template<class InternetProtocol>
    class basic_resolver : public resolver_base
    {
    public:
```

[...]

```
        // 21.17.4, basic_resolver operations:
```

io_context& context() noexcept;

```
        executor_type get_executor() noexcept;
```

[...]

```
explicit basic_resolver(io_context& ctx);
```

-1- **Postconditions:**~~get_executor() == ctx.get_executor();~~

- addressof(context()) == addressof(ctx).
- get_executor() == ctx.get_executor().

```
basic_resolver(basic_resolver&& rhs) noexcept;
```

-2- **Effects:** Move constructs an object of class basic_resolver<InternetProtocol> that refers to the state originally represented by rhs.

-3- **Postconditions:** ~~get_executor() == rhs.get_executor();~~

- addressof(context()) == addressof(rhs.context()).
- get_executor() == rhs.get_executor().

[...]


```
basic_resolver& operator=(basic_resolver&& rhs);
```

-1- **Effects:** Cancels all outstanding asynchronous operations associated with *this as if by calling `cancel()`, then moves into *this the state originally represented by rhs.

-2- **Postconditions:** ~~`get_executor() == ctx.get_executor();`~~

- `addressof(context()) == addressof(rhs.context());`
- `get_executor() == rhs.get_executor();`

-3- **Returns:** *this.

21.17.4 `ip::basic_resolver` operations [internet.resolver.ops]

`io_context& context() noexcept;`

-?- **Returns:** The associated execution context.